

Code review — mini-proiecte multithreading

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-10-08 — sha 24858b8

Revizuit: M03 refacut si M04 (`a6d1e1b`), plus lotul 1-bis — M01b, M02b, M03b, M04b (`ef2263b` .. `24858b8`). M05 si M05b nu sunt incepute.

Poarta de compilare: trece. Toate cele 10 fisiere compileaza cu 0 erori. Am rulat fiecare program de doua ori; tot ce scriu mai jos e reprodus prin rulare.

Ce s-a inchis si ce e bine:

-  **Toate trei**  **din runda 2 sunt inchise, verificat prin rulare.** M03 numara acum comenzile acolo unde se lucreaza (`SUMAR: 20`), le face una dupa alta pe `main` (**2111 ms** si **2381 ms**, cu jurnalul intercalat intre comenzi), iar jurnalul e daemon — programul iese singur si ultima linie e sumarul. Ai renuntat si la `interrupt()` + `join()`. Toata runda, inchisa curat.
-  **M04b trece toate criteriile.** `Supplier gama is out of stock` cu `e.getCause().getMessage()` (`:102`), 2 fire pentru 5 cereri (`pool-1-thread-1` si `-2` apar fiecare de doua ori), **1338 ms** fata de o suma de **2500 ms** si peste cel mai lent (700 ms), iar cel mai ieftin (vodafone) e chiar cel mai lent, nu primul care a raspuns.
-  **M01b:** ordinea iesirii e cea reala si creste (`402` → `754` → `825`), verdict `in paralel`, 826 ms fata de suma 1981 ms. Lista comuna o protejezi cu `synchronized (iesite)` — corect, si din reflex.
-  **M02b:** ai scris `extends Thread`, si totusi ai intrebat `Thread.currentThread().getName()`, nu `getName()`. Exact capcana pe care o ascundea proiectul: la varianta B, `getName()` ar fi raspuns `Thread-0..5` si ar fi parut 6 fire. Tu ai primit `main` de 6 ori → 1 fir.

Critice

B1  `Thread.sleep(100)` e in afara buclei — bara se redeseneaza de 4 milioane de ori

src/M03-jurnal-de-fundal/M03b.java:79

CUM E ACUM: prima rulare a scos **4.018.724** de linii in 2,4 secunde. Intre **Fisier 1 copiat** (linia 260.924) si **Fisier 2 copiat** (linia 572.570) bara s-a redesenat de peste **300.000** de ori — pentru o copiere care dureaza cel mult 200 ms. Cerinta: o redesenare la fiecare 100 ms.

DE CE: uita-te la acolada de pe linia 79:

```
System.out.println("Progress: " + procent + "%");
if(copiate == 15){ return; }
}Thread.sleep(100);
```

} inchide **while** -ul, deci **sleep** nu mai e in corpul buclei — e **dupa** ea. In bucla n-a mai ramas nimic care sa opreasca firul, asa ca el intreaba procentul si il tipareste fara pauza, cat de repede poate procesorul. Asta se numeste *busy-wait*: firul nu asteapta, ci arde un nucleu ca sa afle de milioane de ori acelasi raspuns. Linia **sleep** nu se executa niciodata: bucla se termina doar prin **return**, care sare peste ea.

Programul **pare** sa treaca — iese singur, ultima linie e sumarul, 15/15 — si de-aia e periculos. Criteriile pe care le verifici cu ochiul trec; cel pe care trebuia sa-l numeri, nu.

FIX: **sleep** intra in bucla, la final. Formatarea automata (Ctrl+Alt+L in IntelliJ) ar fi pus **Thread.sleep** pe un rand separat, dupa **}**, si s-ar fi vazut.

B2 🚫 La factura 3 tiparesti exceptia de invelis, nu motivul

src/M04-calculator-facturi/M04.java:94

CUM E ACUM:

```
Factura 3 : java.lang.Exception: Invalid amount on invoice 3
```

Criteriul cere **Invalid amount on invoice 3**, **nu** numele unei exceptii de invelis.

DE CE: exceptia ta e aruncata pe un fir din pool, nu pe **main**. Ea nu poate sari direct dintr-un fir in altul; pool-ul o prinde si o pune in **Future**. Cand chemi **get()**, **Future** arunca o exceptie **noua**, **ExecutionException**, cu a ta pusa inauntru ca *cauza*. Iar mesajul invelisului e construit din cauza cu **toString()**: numele clasei, doua puncte, mesajul. De acolo vine **java.lang.Exception:** din fata.

Fixul il stii deja — l-ai scris singur o saptamana mai tarziu, in **M04b.java:102**:

e.getCause().getMessage(). Ai deschis M04 pe 7 octombrie si ai adaugat **total += rezultate**,

dar n-ai dus inapoi `getCause()`.

FIX: `e.getCause().getMessage()` in loc de `e.getMessage()`.

B3 🚫 M04: lipsesc milisecundele de pe fiecare linie si propozitia ta

`src/M04-calculator-facturi/M04.java:92-94`

CUM E ACUM: cerinta cere pe fiecare linie *“la cate milisecunde de la pornirea programului ai obtinut rezultatul”* si, in output, *“o singura propozitie scrisa de tine”* care explica ritmul liniilor. Nu e niciuna. Am pus eu ceasul pe output-ul tau, din afara programului:

```
1870 ms | Factura 0 total: 245
1870 ms | Factura 1 total: 215
1870 ms | Factura 2 total: 180
2172 ms | Factura 3 : java.lang.Exception: Invalid amount on invoice 3
2172 ms | Factura 4 total: 310
2172 ms | Factura 5 total: 100
```

DE CE (asta e propozitia pe care trebuie s-o scrii): `get()` pe factura 0 blocheaza `main` 1800 ms. In timpul asta facturile 1 si 2 s-au terminat deja (1500 si 1200 ms), deci `get()` -urile lor intorc pe loc. Dar facturile 3, 4, 5 nu aveau fir: pool-ul are 3, si fiecare a asteptat pe rand sa se elibereze unul (la 1200, 1500, 1800 ms), iar duratele lor (900, 600, 300) le-au adus pe toate la ~2100. De-aia vezi **doua valori**, nu unul. Saltul de 300 ms dintre ele e pool-ul din `B4`, vazut in cifre.

FIX: `long acum = System.currentTimeMillis() - start;` dupa fiecare `get()` si in `catch`, tiparit pe linie; propozitia ta, tiparita la final.

B4 🚫 M02b nu spune de cate ori a fost A mai rapid

`src/M02-cronometru-pacalit/M02b.java:99-102`

CUM E ACUM: Varianta A: 427 ms , Varianta B: 2427 ms — si gata. Criteriul cere la final `A a fost de N ori mai rapid` cu N calculat.

DE CE: e exact aceeasi linie ca la M02, unde in runda 1 ai tiparit fractia rasturnata (`0.26`) si ai reparat-o in runda 2. Acum lipseste de tot. Nu e o problema de Java — e partea a doua a cerintei care cade. Vezi mai jos, la *Tiparul rundej*.

FIX: aceeasi formula ca in `M02.java:82` , `durataB / durataA` cu rotunjire — iti iese ~5.7.

Before / after

#	acum	corect
B1	<code>}Thread.sleep(100);</code> dupa bucla	<code>Thread.sleep(100);</code> ultima linie in <code>while</code>
B2	<code>e.getMessage()</code>	<code>e.getCause().getMessage()</code>
B3	<code>"Factura " + i + " total:"</code> <code>" + r</code>	<code>"Factura " + i + " total:"</code> <code>" + r + " la " + acum + "</code> <code>ms"</code>
B4	lipseste	<code>"A a fost de " +</code> <code>Math.round(durataB * 10.0</code> <code>/ durataA) / 10.0 + " ori</code> <code>mai rapid"</code>

Importante

- **M1** 🟡 `M01b.java:92` — raportul cere si **ordinea in care ai pornit** cuptoarele. Tiparesti doar ordinea iesirii.
- **M2** 🟡 `M03b.java:83-86` — la finalul lui `Bara.run()` construiesti si pornesti **o alta** `Bara`. Azi nu se executa (iesi prin `return`), dar daca bara ar fi intrerupta, s-ar inlocui singura, la nesfarsit. Pornirea barei e treaba lui `main` (`:94-96`), si acolo o ai deja.
- **M3** 🟡 `M03b.java:76-78` — bara se opreste singura cand vede 15. Dar cerinta zice ca bara *"nu stie dinainte"* — ea traieste cat aplicatia, iar de oprire se ocupa daemon-ul. La M03 ai facut-o corect (`while (true)`).
- **M4** 🟡 `M03b.java:93-98` — copierea e un fir separat pe care `main` doar il asteapta. Hintul intreba *"cate fire in plus fata de `main`?"*: raspunsul e unul, bara. La M03 ai pus munca direct pe `main` — fa la fel.
- **M5** 🟡 `M02b.java:28,35` — la varianta A, 6 fire scriu in acelasi `HashSet` fara nicio protectie. `add` e citeste-verifica-scrie, ca `contor++` din `C1-race-condition`: doua fire pot pierde un nume. **Nu l-am reprodus** in doua rulari (firele pornesc la microsecunde distanta), dar la M01b ai protejat lista exact pentru asta. Aplica aceeasi regula.

Cleanups

- **C1**  M04b.java:117–118 — `vodafoneeste100` (lipseste spatii), `Oferte esute` . M03b.java:37 — clasa `Copiere` .
- **C2**  M02b.java:85–97 — varianta B nu are `join()` -urile, deci difera de A prin mai mult de un cuvânt. Le puteai lasa: `join()` pe un fir nepornit intoarce imediat.
- **C3**  M04b.java:101 — `catch (Exception)` prinde si `InterruptedException` , care nu are cauza → `getCause()` e `null` si primesti `NullPointerException` . Prinde `ExecutionException` separat.

Ramase deschise din runda 1 — a treia runda neatinsa

#	Unde	Ce
M1 	M02.java:72–75	<code>run()</code> pe fire deja consumate de <code>start() + join()</code>
M2 	M01.java:82	<code>System.out.println("test")</code> in mijlocul raportului
C1 	M01.java:92–93	citesti <code>pagina.durata</code> direct, desi ai getteri
C2 	M02.java:63,79,83	<code>Vaianta</code> ×3, <code>Medtoda</code> , lipsa spatiu: <code>3.8ori</code>

Sunt patru linii, in doua fisiere. Le-ai avut pe lista de doua runde.

Tiparul rundei

M03 arata ca poti inchide o runda **complet**: trei  , toate trei rezolvate, fiecare pana la capat. E prima data.

Pe proiectele noi, a doua jumatate a cerintei a cazut de trei ori:

proiect	jumatatea facuta	jumatatea lipsa
M01b	ordinea iesirii	ordinea pornirii
M02b	durate + fire distincte	de N ori mai rapid
M04	totalul fiecărei facturi	milisecundele + propozitia

Înainte să dai commit, citește **criteriul de acceptare** din headerul fisierului rând cu rând și găsește fiecare lucru cerut în output-ul tău. Dacă nu-l poți arăta cu degetul, nu e gata.

Întrebări de verificare

1. În M03b, linia `Thread.sleep(100)` există în cod. De ce nu se execută niciodată — și cum ai fi aflat asta doar din numărul de linii din output?
2. În M04 și M04b ai scris două `catch`-uri aproape identice: unul tipărește `java.lang.Exception: Invalid ...`, celălalt doar mesajul. Cine a construit textul `java.lang.Exception:` — și de ce el, nu tu?
3. În M04 liniile vin în **două** valuri (1870 ms și 2172 ms). Dacă pool-ul ar avea 6 fire în loc de 3, câte valuri ar fi? Dar cu 1 fir?

Următorul pas: cele 4 🚫 de mai sus (trei fișiere), cele patru rămase din runda 1, apoi M05 și M05b.

Runda 2 — 2026-10-01 — sha 275b6cb

M03 — trei 🚫, toate în `M03.java`.

- ☒ **B1 închis** (runda 3) — contorul creștea în jurnal (`getAndIncrement`), nu în comandă → `SUMAR: 1`. Acum `SUMAR: 20`.
- ☒ **B2 închis** (runda 3) — comenzile porneau în paralel (205 ms în loc de ~2000). Acum sunt pe `main`, una după alta: 2111 ms.
- ☒ **B3 închis** (runda 3) — jurnalul nu era daemon și era oprit cu `interrupt()` + `join()`. Acum `setDaemon(true)` înainte de `start()`.

Runda 1 — 2026-09-29 — sha d845a33

M01 si M02, livrate la mai putin de o ora dupa primirea lotului. Poarta de compilare: trece.

-  **B1 inchis** (runda 2) — `M02.java:82` , verdictul tiparea fractia rasturnata: `0.26` in loc de `3.8` .
-  **M1 NEATINS** — `M02.java:72-75` , `run()` pe fire deja pornite si terminate. Merge doar fiindca `run()` e metoda obisnuita; `start()` a doua oara ar da `IllegalThreadStateException` .
-  **M2 NEATINS** — `M01.java:82` , `println("test")` in mijlocul raportului.
-  **C1 NEATINS** — `M01.java:92-93` , getteri scrisi si apoi ocoliti prin acces direct la camp.
-  **C2 NEATINS** — `Vaianta` ×3, `Medtoda` , lipsa spatiu la `3.8ori` .

Bine in runda 1: la M01 `join()` in bucla **separata** de `start()` (`:77` si `:80`) — miezul proiectului; verdict `in paralel` , 1061 ms fata de o suma de 3832 ms. La M02, diferenta de **un singur cuvant** intre variante (`start()` la 50, `run()` la 72), confirmata de numele firelor.